

Lógica Computacional 2024/2025 - TP3

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: !pip install pysmt
!pip install z3-solver
!pysmt-install --msat
```

```
In [ ]: from pysmt.shortcuts import *
from pysmt.typing import *
import itertools
```

Exercício 1

Enunciado

O algoritmo estendido de Euclides (EXA) aceita dois inteiros constantes $a, b > 0$ e devolve inteiros r, s, t tais que $a * s + b * t = r$ e $r = \gcd(a, b)$.

Para além das variáveis r, s, t o código requer 3 variáveis adicionais r', s', t' que representam os valores de r, s, t no “próximo estado”.

```
INPUT  a, b
assume a > 0 and b > 0
r, r', s, s', t, t' = a, b, 1, 0, 0, 1

0: while r' != 0
1:   q = r div r'
2:   r, r', s, s', t, t' = r', r - q * r', s', s - q * s', t', t - q * t'
3: stop

OUTPUT r, s, t
```

a. Construa um SFOTS usando BitVector's de tamanho n que descreva o comportamento deste programa. Considere estado de erro quando $r = 0$ ou alguma das variáveis atinge o “overflow”.

b. Prove, usando a metodologia dos invariantes e interpolantes, que o modelo nunca atinge o estado de erro.

Para simplificar o uso de r', s' e t' nas resoluções que se seguem, estes serão referidos como rl, sl e tl , respetivamente.

a. Construa um SFOTS usando BitVector's de tamanho n que descreva o comportamento deste programa. Considere estado de erro quando $r = 0$ ou alguma das variáveis atinge o “overflow”.

Para modelar este programa como um SFOTS teremos o conjunto X de variáveis do estado dado pela lista

```
['r', 'rl', 's', 'sl', 't', 'tl', 'q', 'pc']
```

Para definir as variáveis do modelo é definida a função `genState` que recebe a lista com o nome das variáveis do estado, uma etiqueta, um inteiro e o número de bits, e cria a i -ésima cópia das variáveis do estado para essa etiqueta. As variáveis lógicas começam sempre com o nome de base das variáveis dos estado, seguido do separador `!`.

```
In [ ]: def genState(vars, s, i, nbits):
state = {}
for var in vars:
state[var] = Symbol(f"{var}!{s}_{i}", BVType(nbits))
return state
```

Para definir o estado inicial é definida a função `init` que, dado um possível estado do programa (um dicionário de variáveis), dois inteiros a e b , e o número de bits, devolve um predicado do pySMT que testa se $r = a, rl = b, s = 1, sl = 0, t = 0, tl = 1, q = 0, a > 0, b > 0$ são todos satisfazíveis, isto é, se esse estado é um possível estado inicial do programa.

```
In [ ]: def init(state, a, b, nbits):

r = Equals(state['r'], BV(a, nbits))
rl = Equals(state['rl'], BV(b, nbits))
s = Equals(state['s'], BV(1, nbits))
sl = Equals(state['sl'], BV(0, nbits))
t = Equals(state['t'], BV(0, nbits))
```

```

tl = Equals(state['tl'], BV(1, nbits))
pc = Equals(state['pc'], BV(0, nbits))
q = Equals(state['q'], BV(0, nbits))

apos = BVSGT(BV(a, nbits), BV(0, nbits))
bpos = BVSGT(BV(b, nbits), BV(0, nbits))

return And(apos, bpos, r, rl, s, sl, t, tl, pc, q)

```

Para definir os estados de erro é definida a função `error` que, dado um estado do programa, devolve um predicado do pySMT que testa se esse estado é um possível estado de erro do programa. Como r , s e t guardam os valores de rl , sl e tl , respetivamente, não é necessário criar um estado de erro no caso de overflow para essas variáveis.

Como o `pysmt` não aceita inputs negativos, mesmo com "signed" BitVectors (já que é usado o complemento para dois), é necessário usar `signPraUnsign`, que converte valores "signed" o seu "unsigned" equivalente. Desta forma é assegurado que todos os valores BV são válidos, que `MAX_VAL` e `MIN_VAL` representam corretamente o intervalo válido para valores "signed", e que, consequentemente, o `overflow` seja detetado corretamente.

```

In [ ]: def signPraUnsign(value, nbits):
        #Converte um signed para seu equivalente unsigned
        return value % (2 ** nbits)

def error(state, nbits):
    #Define valores max and min para inteiros signed com nbits
    MAX_VAL = BV(signPraUnsign((1 << (nbits - 1)) - 1, nbits), nbits) # 2^(n-1) - 1
    MIN_VAL = BV(signPraUnsign(-(1 << (nbits - 1)), nbits), nbits)    # -2^(n-1)

    #r=0
    er0 = Equals(state['r'], BV(0, nbits))

    #Overflow em rl
    rl_sub = BSub(state['r'], BVMul(state['q'], state['rl']))
    erl = And(Equals(state['pc'], BV(2, nbits)),
              Or(BVSLT(rl_sub, MIN_VAL), BVSGT(rl_sub, MAX_VAL)))

    #Overflow em sl
    sl_sub = BSub(state['s'], BVMul(state['q'], state['sl']))
    esl = And(Equals(state['pc'], BV(2, nbits)),
              Or(BVSLT(sl_sub, MIN_VAL), BVSGT(sl_sub, MAX_VAL)))

    #Overflow em tl
    tl_sub = BSub(state['t'], BVMul(state['q'], state['tl']))
    etl = And(Equals(state['pc'], BV(2, nbits)),
              Or(BVSLT(tl_sub, MIN_VAL), BVSGT(tl_sub, MAX_VAL)))

    return Or(er0, erl, esl, etl)

```

Para definir as relações de transição é definida a função `trans` que, dados dois estados do programa, devolve um predicado do pySMT que testa se é possível transitar do primeiro para o segundo estado.

```

In [ ]: def trans(curr, prox, nbits):

    t01 = And(Equals(curr['pc'], BV(0,nbits)),
              Not(Equals(curr['rl'], BV(0,nbits))),
              Equals(prox['pc'], BV(1,nbits)),
              Equals(prox['r'], curr['r']),
              Equals(prox['rl'], curr['rl']),
              Equals(prox['s'], curr['s']),
              Equals(prox['sl'], curr['sl']),
              Equals(prox['t'], curr['t']),
              Equals(prox['tl'], curr['tl']),
              Equals(prox['q'], curr['q']))

    t12 = And(Equals(curr['pc'], BV(1,nbits)),
              Equals(prox['pc'], BV(2,nbits)),
              Equals(prox['q'],BVSDiv(curr['r'], curr['rl'])),
              Equals(prox['r'], curr['r']),
              Equals(prox['rl'], curr['rl']),
              Equals(prox['s'], curr['s']),
              Equals(prox['sl'], curr['sl']),
              Equals(prox['t'], curr['t']),
              Equals(prox['tl'], curr['tl']))

    t20 = And(Equals(curr['pc'], BV(2,nbits)),
              Equals(prox['pc'], BV(0,nbits)),
              Equals(prox['r'], curr['rl']),
              Equals(prox['rl'], BVSub(curr['r'], BVMul(curr['q'], curr['rl']))),
              Equals(prox['s'], curr['sl']),
              Equals(prox['sl'], BVSub(curr['s'], BVMul(curr['q'], curr['sl']))),
              Equals(prox['t'], curr['tl']),
              Equals(prox['tl'], BVSub(curr['t'], BVMul(curr['q'], curr['tl']))),
              Equals(prox['q'], curr['q']))

    t03 = And(Equals(curr['pc'], BV(0,nbits)),

```

```

    Equals(curr['rl'], BV(0,nbits)),
    Equals(prox['pc'], BV(3,nbits)),
    Equals(prox['r'], curr['r']),
    Equals(prox['rl'], curr['rl']),
    Equals(prox['s'], curr['s']),
    Equals(prox['sl'], curr['sl']),
    Equals(prox['t'], curr['t']),
    Equals(prox['tl'], curr['tl']),
    Equals(prox['q'], curr['q']))

```

```

return Or(t01, t12, t20, t03)

```

Usamos genTrace para gerar um possível estado de execução com n transições.

```

In [ ]: def genTrace(vars,init,trans,error,n,a,b,nbits):
    with Solver() as s:
        X = [genState(vars,'X',i,nbits) for i in range(n+1)] # cria n+1 estados (com etiqueta X)
        I = init(X[0],a,b,nbits)
        Tks = [ trans(X[i], X[i+1], nbits) for i in range(n) ]
        E = [error(X[i], nbits) for i in range(n+1)]

        if s.solve([I, And(Tks), Not(Or(E))]):
            for i in range(n+1):
                print(f"Estado: {i}")
                for v in X[i]:
                    val = s.get_value(X[i][v])
                    unsigned_val = val.constant_value()
                    signed_val = unsigned_val if unsigned_val < (1 << (nbits - 1)) else unsigned_val - (1 << nbits)
                    print(f"          {v} = {signed_val}")
            return True
        else:
            print("Erro")
            return False

```

Resultados

```

In [ ]: vars = ['pc', 'r', 'rl', 's', 'sl', 't', 'tl', 'q']

```

```

In [ ]: a = 3
        b = 2
        nbits = 8
        n = 7

        resultado = genTrace(vars, init, trans, error, n, a, b, nbits)

```

```

Estado: 0
    pc = 0
    r = 3
    rl = 2
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 0
Estado: 1
    pc = 1
    r = 3
    rl = 2
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 0
Estado: 2
    pc = 2
    r = 3
    rl = 2
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 1
Estado: 3
    pc = 0
    r = 2
    rl = 1
    s = 0
    sl = 1
    t = 1
    tl = -1
    q = 1
Estado: 4
    pc = 1
    r = 2
    rl = 1
    s = 0
    sl = 1
    t = 1
    tl = -1
    q = 1
Estado: 5
    pc = 2
    r = 2
    rl = 1
    s = 0
    sl = 1
    t = 1
    tl = -1
    q = 2
Estado: 6
    pc = 0
    r = 1
    rl = 0
    s = 1
    sl = -2
    t = -1
    tl = 3
    q = 2
Estado: 7
    pc = 3
    r = 1
    rl = 0
    s = 1
    sl = -2
    t = -1
    tl = 3
    q = 2

```

```

In [ ]: a = 1
        b = 1
        nbits = 8
        n = 4

        resultado = genTrace(vars, init, trans, error, n, a, b, nbits)

```

```

Estado: 0
    pc = 0
    r = 1
    rl = 1
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 0
Estado: 1
    pc = 1
    r = 1
    rl = 1
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 0
Estado: 2
    pc = 2
    r = 1
    rl = 1
    s = 1
    sl = 0
    t = 0
    tl = 1
    q = 1
Estado: 3
    pc = 0
    r = 1
    rl = 0
    s = 0
    sl = 1
    t = 1
    tl = -1
    q = 1
Estado: 4
    pc = 3
    r = 1
    rl = 0
    s = 0
    sl = 1
    t = 1
    tl = -1
    q = 1

```

b. Prove, usando a metodologia dos invariantes e interpolantes, que o modelo nunca atinge o estado de erro.

Para auxiliar na implementação deste algoritmo, começamos por definir cinco funções.

Para que a prova seja arbitrária é criado um novo init chamado *initB* onde *a* e *b* se tornam valores arbitrários.

```

In [ ]: def initB(state, nbits):

    a = Symbol("a", BVType(nbits))
    b = Symbol("b", BVType(nbits))

    apos = BVSGT(a, BV(0, nbits))
    bpos = BVSGT(b, BV(0, nbits))

    r = Equals(state['r'], a)
    rl = Equals(state['rl'], b)
    s = Equals(state['s'], BV(1, nbits))
    sl = Equals(state['sl'], BV(0, nbits))
    t = Equals(state['t'], BV(0, nbits))
    tl = Equals(state['tl'], BV(1, nbits))
    pc = Equals(state['pc'], BV(0, nbits))
    q = Equals(state['q'], BV(0, nbits))

    return And(apos, bpos, r, rl, s, sl, t, tl, pc, q)

```

A função *invert* recebe a função que codifica a relação de transição e devolve a relação de transição inversa.

A função *baseName* recebe uma string *s* e retorna a parte da string até à primeira ocorrência do carácter '!'.

A função *rename* renomeia uma fórmula (sobre um estado) de acordo com um dado estado.

A função *same* testa se dois estados são iguais.

```

In [ ]: def invert(trans):
    return lambda curr, prox, nbits: trans(prox, curr, nbits)

```

```

def baseName(s):
    return ''.join(list(itertools.takewhile(lambda x: x!='!', s)))

def rename(form, state):
    vs = get_free_variables(form)
    pairs = [ (x, state[baseName(x.symbol_name())]) for x in vs ]
    return form.substitute(dict(pairs))

def same(state1, state2):
    return And([Equals(state1[x], state2[x]) for x in state1])

```

De forma a provar que o modelo não atinge o estado de erro é usada a função *model_checking* que dada a lista de nomes das variáveis do sistema, um predicado que testa se um estado é inicial, um predicado que testa se um par de estados é uma transição válida, um predicado que testa se um estado é de erro, e dois números positivos N e M que são os limites máximos para os índices *n* e *m*.

```

In [ ]: def model_checking(vars, initB, trans, error, N, M, nbits):
    with Solver(name="z3") as solver:

        # Criar todos os estados que poderão vir a ser necessários.
        X = [genState(vars, 'X', i, nbits) for i in range(N+1)]
        Y = [genState(vars, 'Y', i, nbits) for i in range(M+1)]
        transt = invert(trans)

        # Estabelecer a ordem pela qual os pares (n,m) vão surgir. Por exemplo:
        order = sorted([(a,b) for a in range(1,N+1) for b in range(1,M+1)], key=lambda tup: tup[0]+tup[1])

        # Step 1 implícito na ordem de 'order' e nas definições de Rn, Um.
        for (n,m) in order:
            # Step 2.
            I = initB(X[0], nbits)
            Tn = And([trans(X[i], X[i+1], nbits) for i in range(n)])
            Rn = And(I, Tn)

            E = error(Y[0], nbits)
            Bm = And([transt(Y[i], Y[i+1], nbits) for i in range(m)])
            Um = And(E, Bm)

            Vnm = And(Rn, same(X[n], Y[m]), Um)
            if solver.solve([Vnm]):
                print("> 0 sistema é inseguro.")
                return
            else:
                # Step 3.
                A = And(Rn, same(X[n], Y[m]))
                B = Um
                C = binary_interpolant(A, B)

                # Salvar cálculo bem-sucedido do interpolante.
                if C is None:
                    print("> 0 interpolante é None.")
                    break

                # Step 4.
                C0 = rename(C, X[0])
                T = trans(X[0], X[1], nbits)
                C1 = rename(C, X[1])

                if not solver.solve([C0, T, Not(C1)]):
                    # C é invariante de T.
                    print("> 0 sistema é seguro.")
                    return
                else:
                    # Step 5.1.
                    S = rename(C, X[n])
                    while True:
                        # Step 5.2.
                        T = trans(X[n], Y[m], nbits)
                        A = And(S, T)
                        if solver.solve([A, Um]):
                            print("> Não foi encontrado majorante.")
                            break
                        else:
                            # Step 5.3.
                            C = binary_interpolant(A, Um)
                            Cn = rename(C, X[n])
                            if not solver.solve([Cn, Not(S)]):
                                # Step 5.4.
                                # C(Xn) -> S é tautologia.
                                print("> 0 sistema é seguro.")
                                return
                            else:
                                # Step 5.5.
                                # C(Xn) -> S não é tautologia.
                                S = Or(S, Cn)

        print("> Não foi provada a segurança ou insegurança do sistema.")

```

```
varsB = ['pc', 'r', 'rl', 's', 'sl', 't', 'tl', 'q']  
model_checking(varsB, initB, trans, error, 50, 50, 8)
```

```
> Não foi encontrado majorante.  
> O sistema é seguro.
```